

## **Goal 2 Progress Report**

Implementation of Explainable Price Estimation  
and Market Benchmarking Engine

April 9, 2026

## Introduction

This report documents the implementation progress of Goal 2 within the context of the broader project architecture. Goal 2 represents the development of an explainable price estimation and market benchmarking engine for short-term rental listings. However, this goal cannot be evaluated in isolation, as its implementation relies directly on the analytical data foundation established in Goal 1, and its outputs are designed for operationalization through Goals 3 and 4.

The project has evolved from a data-preparation repository into a functioning analytical system with modeling capabilities. The current state includes a complete pipeline for assembling modeling inputs, generating feature matrices, training and comparing models, retrieving comparable listings, computing benchmark ranges, classifying price positioning, generating interpretable local explanations, and packaging results into reusable inference bundles.

The current reference version is **goal2\_v4**, which implements a benchmark-led decision framework.

## Initial Repository State

At the commencement of Goal 2 implementation, the repository possessed a robust analytical foundation. The cleaned datasets were available, data contracts were documented, and cleaning and validation logic were operational. Goal 1 was approaching completion in practical terms.

However, the repository lacked the decision-support components specified in the project plan. Specifically, the following elements were absent:

- No dedicated Goal 2 modeling package
- No supervised estimation engine
- No comparable listings engine
- No benchmark-range calculation layer
- No inference bundle infrastructure
- No application-facing payload contract for pricing decisions

Additionally, documentation alignment issues existed, with several AGENTS.md files referencing outdated repository paths requiring synchronization with the actual workspace configuration.

## Goal 2 Architectural Definition

Prior to implementation, Goal 2 was explicitly defined as a hybrid decision-support engine rather than a conventional machine learning regression system. This architectural decision reflects both academic rigor and practical product requirements.

A pure regression model would produce a single predicted price value, which, while straightforward to implement, would lack interpretability and decision-support utility. Conversely, a comparables-only approach would maximize interpretability but sacrifice analytical sophistication and alignment with project specifications.

The adopted architecture comprises three coordinated components:

- 1. Supervised Price Estimation Model** — provides quantitative price predictions
- 2. Comparable Listings Engine** — identifies locally similar properties
- 3. Benchmark Range Layer** — computes market positioning with fallback logic

This architecture was formally documented in planning and scope documents, with explicit non-goals defined to prevent scope expansion:

- No occupancy prediction capabilities
- No revenue management functionality
- No demand elasticity modeling
- No dynamic pricing algorithms
- No backend or frontend serving infrastructure in this phase

## Modeling Core Implementation

Following scope definition, the repository received a comprehensive Goal 2 implementation. This phase represented the most substantial development effort in the project to date.

The implementation introduced modular feature areas within `modeling/src/`:

- `feature_engineering/` — feature construction and transformation
- `benchmarking/` — market comparison logic
- `price_estimation/` — supervised learning models
- `explainability/` — interpretable output generation
- `evaluation/` — model assessment metrics

## Data Processing Pipeline

The implementation begins with cleaned Goal 1 datasets and constructs a canonical listing-level modeling table. This table serves as the foundation for feature matrix generation and model training.

Two estimators are trained for comparative evaluation:

**Linear Baseline** — ElasticNet regression providing interpretable coefficients

**Tree Challenger** — HistGradientBoostingRegressor capturing non-linear relationships

The system implements grouped splitting by `listing_id`, preventing the same listing from appearing in both training and test sets across different temporal snapshots. This design choice protects against data leakage, a critical concern in panel-style listing datasets.

## Decision Support Components

Complementing the supervised estimation layer, a comparable listings engine identifies locally similar properties based on geographic proximity and property characteristics. A benchmark range calculator derives market positioning from either comparable listings or, when necessary, fallback summary statistics.

A price positioning service integrates all components into a unified decision payload containing:

- Benchmark range (market positioning bounds)
- Price positioning label (competitive classification)
- Comparable listings (market reference set)
- Model estimate (predicted price)
- Local explanation (feature attribution)

## Integration Contracts and Documentation

To facilitate future integration with backend and frontend layers, shared YAML contracts were established under `shared/contracts/modeling/`:

- `price_decision_payload.yaml` — decision output specification
- `comparable_listing.yaml` — market reference structure
- `model_bundle_metadata.yaml` — artifact versioning schema

Supporting documentation was created under `docs/modeling/`, including model input specifications, feature catalogs, and handoff documentation.

These contracts establish a structured interface between the modeling layer and future application components, preventing Goal 2 from remaining isolated within notebooks or

internal scripts. With explicit contracts, backend systems can reliably load, validate, and expose modeling outputs.

## Testing and Quality Assurance

Goal 2 functionality was implemented with comprehensive test coverage. The `modeling/` module currently passes the complete pytest suite with the following metrics:

**Tests Passing:** 51

**Code Coverage:** 93.20%

This coverage level demonstrates that Goal 2 code is actively exercised through automated testing and exceeds the project-wide minimum coverage threshold of 85%. The testing infrastructure provides confidence in the reliability and maintainability of the implementation.

## Version Evolution and Refinement

The Goal 2 implementation progressed through four distinct versions, each addressing specific limitations identified through evaluation and analysis.

### goal2\_v1: Initial Implementation

The initial version established the foundational architecture with all core components operational. However, evaluation revealed susceptibility to extreme outliers in the training data, resulting in degraded prediction quality.

### goal2\_v2: Data Quality Enhancement

Version 2 implemented systematic removal of implausible outliers through domain-informed filtering rules. This data quality improvement yielded substantial performance gains, demonstrating that preprocessing refinements can produce more significant improvements than architectural modifications.

### goal2\_v3: Explanation Enhancement

Version 3 focused on improving the interpretability of model outputs through enhanced local explanation generation. While predictive metrics remained unchanged, usability substantially improved. This version demonstrated that for decision-support systems, interpretability and stakeholder trust constitute essential product attributes rather than secondary considerations.

### goal2\_v4: Benchmark-Led Framework

The current reference version implements a semantic refinement in output presentation. Analysis across versions consistently demonstrated that benchmark ranges and comparable listings provide the most defensible signals, while point estimates, though useful, remain secondary.

Version 4 restructures the payload to explicitly communicate:

- `benchmark_range` as the primary decision signal
- `benchmark_led_positioning` as the decision policy
- Point estimate as a supporting signal

This framing does not improve numerical performance metrics but enhances semantic honesty by accurately representing the relative reliability of different signals within the system. Version 4 serves as the reference implementation not due to superior predictive accuracy, but because it more accurately communicates the system's actual strengths.

## Current Project State

The project has achieved substantial progress from its initial state. Goal 1 is effectively complete, with all data cleaning, validation, provenance handling, and contract specifications operational and reproducible.

Goal 2 is implemented at the modeling layer with the following capabilities:

- Canonical model input assembly
- Reusable feature engineering pipeline
- Supervised training with model comparison
- Comparable listing retrieval system
- Benchmark range calculation
- Price positioning classification
- Local explanation generation
- Versioned artifact management
- Reusable inference bundle packaging

However, Goal 2 lacks application serving infrastructure. No API exists to load the inference bundle at runtime, no request/response layer is available for users or external services, and no dashboard or user interface supports practical deployment. The analytical core is complete, but operationalization remains pending.

## Key Technical Insights

Several important conclusions emerged from the implementation process:

- 1. Data Foundation Quality** — The Goal 1 analytical base proved sufficiently robust to support Goal 2 implementation without requiring reconstruction.
- 2. Benchmark Layer Strength** — Comparable listings and benchmark ranges demonstrate superior interpretability and decision-support utility, establishing the benchmark range as the system's primary signal.
- 3. Point Estimate Limitations** — While the model estimate provides valuable information, it should not be presented as the primary signal. Performance on upper price bands requires further improvement.
- 4. Data Quality Impact** — Systematic outlier removal (v1 to v2 transition) produced more substantial performance improvements than architectural refinements would likely have achieved.
- 5. Explanation Quality Importance** — Enhanced interpretability (v2 to v3 transition) significantly improved system usability despite unchanged predictive metrics, confirming that legibility and trust are core product attributes for decision-support systems.
- 6. Output Framing Significance** — Benchmark-led framing (v4) improved system communication without changing numerical performance, demonstrating the importance of honest signal representation.

## Remaining Implementation Gaps

Despite Goal 2 implementation, several critical gaps prevent full operational deployment:

**Backend Serving Infrastructure** — No FastAPI application exists to load the inference bundle and respond to requests in a contract-driven manner.

**User Interface Layer** — No analyst-facing dashboard or interactive workflow exists for exploring listings, reviewing benchmarks, examining comparables, and interpreting explanations.

**Point Estimate Robustness** — The benchmarking layer demonstrates credibility, but the supervised model requires further refinement, particularly for high-value listings.

**Productization Discipline** — While versioned artifacts exist, operational conventions for model releases, inference input validation, and serving behavior require formalization.

## Recommended Development Path

The subsequent development phase should proceed according to the following prioritized sequence:

### Priority 1: Goal 2 Refinement

Before integration, Goal 2 should be strengthened based on empirical evidence:

- Revisit champion model selection policy, as tree-based models demonstrate superior accuracy despite reduced interpretability
- Improve robustness for upper price band listings
- Evaluate target transformation strategies (log1p, controlled winsorization)
- Refine explanation layer to ensure consistent business-readable feature attribution

## Priority 2: Goal 3 Backend Integration

Develop a FastAPI layer that loads the serialized inference bundle, validates inputs, maps requests to the Goal 2 feature schema, and returns stable responses containing benchmark ranges, positioning labels, comparable listings, explanations, and supporting model estimates.

## Priority 3: Goal 4 User Interface

Following backend implementation, develop an analyst-facing dashboard presenting information in order of analytical strength: benchmark range first, comparable listings second, model estimate third, and local explanation fourth.

## Priority 4: Final Productization

Establish model cards, artifact release conventions, inference validation protocols, and demonstration scenarios to support disciplined system maintenance and evolution.

## Recommended Academic Positioning

For presentation to academic supervisors, reviewers, or stakeholders, the following positioning statement accurately reflects the current project state:

*The project has completed its analytical data foundation and implemented a functional Goal 2 pricing decision-support engine. This engine integrates supervised price estimation, local market benchmarking, and comparable listing retrieval. The benchmark-driven positioning logic represents the system's strongest and most reliable component, which the latest payload version (goal2\_v4) explicitly reflects by treating the benchmark range as the primary decision signal. The next development stage focuses on operationalizing the engine through backend service integration and user-facing dashboard development.*

## Conclusion

The most significant achievement to date is the transformation of the repository from an analytical preparation project into a system containing a functional and reusable product core.

The project now possesses:

- A reproducible Goal 2 analytical pipeline
- Comprehensive test coverage for analytical logic
- Versioned modeling artifacts with clear provenance
- Explainable pricing decision payloads
- Defined integration path for backend and frontend development

This represents substantial progress. The central question is no longer whether Goal 2 can be implemented—implementation is complete. The current challenge is to refine the implementation where evidence indicates weaknesses and to transform the analytical core into an operational application through Goals 3 and 4.